

Advanced Flutter Development

WebSockets, Permissions, and Routing

Part 1: The Real-Time Connection

Flutter & WebSockets

The Problem with HTTP for Real-Time

The HTTP Model

HTTP operates on a "request-response" model. The client (your app) asks for data, and the server sends a single response. After the response, the connection is done. The server cannot "push" new data to the client.

The "Polling" Problem

To simulate real-time, apps must repeatedly ask, or "poll," the server (e.g., "Any new messages? ... Any new messages?"). This is inefficient, creates high network traffic, increases server load, and introduces latency.

The WebSocket Solution



Persistent

The connection is opened once via a "handshake" and stays open, eliminating the overhead of new requests.



Full-Duplex

Once open, both the client and server can send messages at any time, without waiting for a "request."



Server-Push

The server can actively "push" data to the client, enabling true low-latency, real-time experiences.

HTTP vs. WebSockets

Feature	HTTP (with Keep-Alive)	WebSockets
Connection Model	Transactional (Request-Response)	Persistent, Stateful
Data Flow	Half-Duplex (Client requests)	Full-Duplex (Client & Server send)
Server-Push?	No (Requires client polling)	Yes (True server-push)
Use Cases	REST APIs, static content	Live chat, real-time feeds, gaming

The WebSocket Handshake

- 1 Client Request:** The client sends a standard HTTP GET request, but with special headers: Upgrade: websocket and Connection: Upgrade.
- 2 Server Response:** A WebSocket-aware server recognizes this and replies with an HTTP 101 Switching Protocols status code.
- 3 Connection Upgraded:** The handshake is complete. The same TCP connection is "hijacked" and becomes the persistent, full-duplex WebSocket.

Flutter: web_socket_channel

The official `web_socket_channel` package provides an elegant abstraction for WebSockets in Dart.

1. The Stream

`channel.stream` is a `Stream` of messages **from** the server. You listen to this to receive data.

2. The Sink

`channel.sink` is a `StreamSink` for messages **to** the server. You add to this to send data.



Code: Simple Echo Client

Listen with StreamBuilder

The StreamBuilder widget is the perfect tool. It listens to channel.stream and rebuilds the UI every time a new message arrives.

```
StreamBuilder(  
  stream: _channel.stream,  
  builder: (context, snapshot) {  
    return Text(snapshot.hasData  
      ? '${snapshot.data}'  
      : 'No data yet');  
  },  
)
```

Send with the Sink

To send a message, you simply call .add() on the sink with your data. This pushes it to the server instantly.

```
void _sendMessage() {  
  if (_controller.text.isNotEmpty) {  
    _channel.sink.add(  
      _controller.text  
    );  
  }  
}
```


Critical Best Practice: Dispose!

“ A WebSocket is a persistent connection. If `channel.sink.close()` is not called in the `dispose()` method, it creates a resource leak (memory, battery, and network).

— This is a mandatory cleanup step for any stateful widget using WebSockets. ”



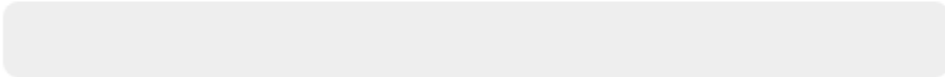
Part 2: Managing Device Access

The Flutter Permission
Model

Step 1: Static Configuration

Android: AndroidManifest.xml

You must declare your **intent** to use a permission.
This tells the OS what your app **might** ask for.



iOS: Info.plist

You must provide the **exact string** that will be shown to the user. If this is missing, your app will crash.

```
NSCameraUsageDescription  
This app needs the camera  
to take profile photos.
```

Step 2: Runtime Handling with `permission_handler`



isGranted

You're in. Proceed with the feature.



isDenied

User said no, but you can ask again. Show a rationale first!



isPermanentlyDenied

A "one-way gate." User checked "Don't Ask Again." You **cannot** request.



isRestricted

User cannot grant (e.g., Parental Controls). Disable the feature.

The "isPermanentlyDenied" Trap

“ This is a one-way gate. The request() method stops working. The only "key" to unlock it is the openAppSettings() function, which cedes control entirely to the user. ”

— This is the most common failure point for developers handling permissions.

Permission UX Best Practices



Ask Contextually: Don't ask on launch. Request the permission at the **exact** moment the user tries to use the feature.



Explain Why (Rationale): Show a friendly in-app dialog **before** the scary system prompt. This builds trust and increases "Allow" rates.



Handle Denial Gracefully: Don't crash or block the app. "Gracefully degrade" the feature (e.g., offer upload from gallery if camera is denied).



Master the 'Denied' Flow: If `isPermanentlyDenied`, show a button that calls `openAppSettings()` to give the user a path to recovery.



Part 3: Navigating the App

Flutter Routing from Basics to
Pro

Navigator 1.0: The (Old) Imperative Way

Anonymous Routes

The most basic form. You "push" a `MaterialPageRoute` that builds your widget. This is simple but leads to duplicated code.

```
Navigator.push(  
  context,  
  MaterialPageRoute(  
    builder: (ctx) => SecondScreen()  
  ),  
);
```

Named Routes

Cleaner. You define a map of string names to widgets in `MaterialApp`. However, this is no longer recommended.

```
// 1. Define in MaterialApp  
routes: {  
  '/second': (ctx) => SecondScreen(),  
}  
  
// 2. Navigate by name  
Navigator.pushNamed(context, '/second');
```


Limitations of Navigator 1.0

The official Flutter docs warn against using named routes. Why?



Inflexible Deep Linking: Poorly handles external links (e.g., from a notification) trying to open a specific screen.



Difficult Stack Management: You can only push/pop from the top. Replacing the entire stack (e.g., after login) is complex.



Poor Web Support: Doesn't map to browser URLs. The back/forward buttons and URL bar don't work correctly.

The Solution: go_router

go_router is the official, Flutter-team-endorsed package that implements the powerful, declarative **Navigator 2.0** API for you.

Declarative: "Stack = f(State)"

You no longer say "go here." You define your app's navigation state, and the router builds the stack.

URL-Based API

It provides a simple, URL-based API (e.g., `context.go('/users/123')`) that solves deep linking, stack management, and web support.



go_router: Passing Parameters

Method	Path Definition	Navigation Example	Use Case
Path Parameter	'/users/:id'	context.go('/users/123')	Bookmarkable. Required IDs.
Query Parameter	'/search'	context.go('/search?q=flutter')	Bookmarkable. Optional filters.
Extra Parameter	(None)	context.go('...', extra: myObject)	NOT Bookmarkable. For transient state.

go_router Best Practice

Pass **IDs** in your routes, not complex objects. Use the ID in the destination screen to look up the data from a central state manager (e.g., Riverpod, Bloc).

— This ensures your app is fully deep-linkable, bookmarkable, and restorable.

Nested Nav: ShellRoute

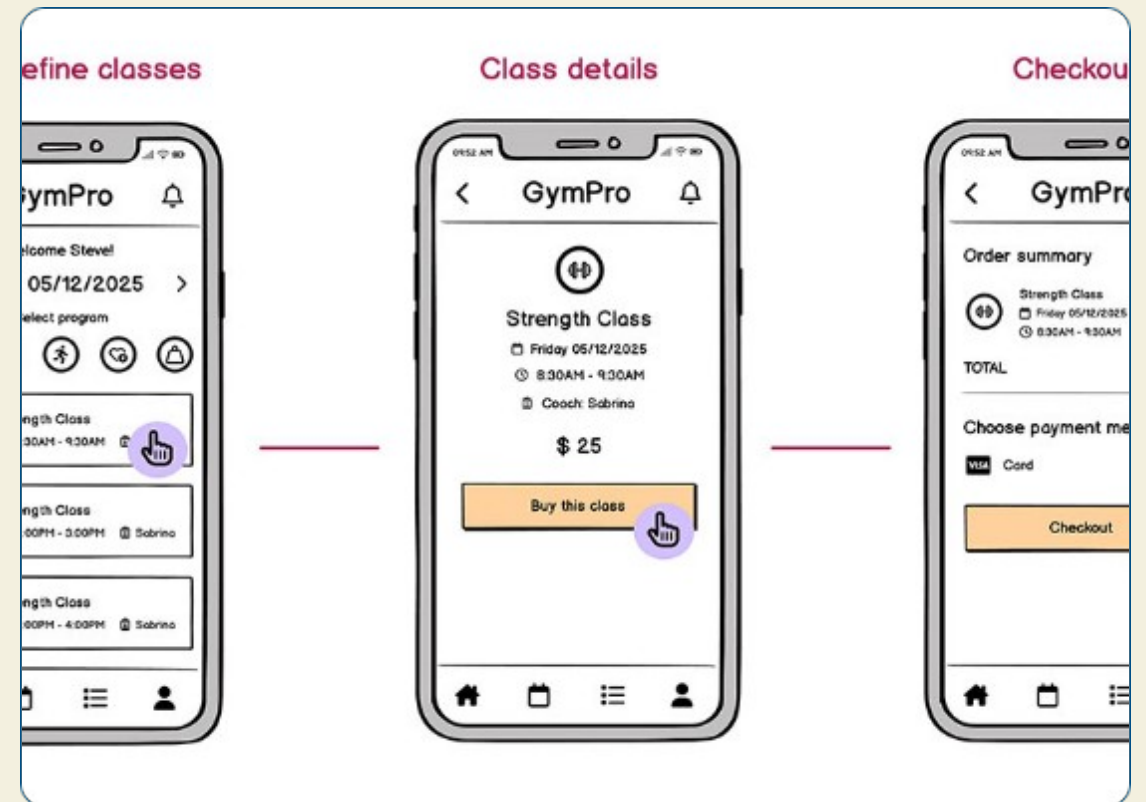
go_router's elegant solution to a classic problem: a persistent BottomNavigationBar.

How it Works

ShellRoute creates a nested Navigator. This means your main Scaffold (with the BottomNavBar) never rebuilds. Only the "shell" body changes.

State Preservation

StatefulShellRoute preserves the navigation state of each tab. If you navigate deep in Tab A, switch to Tab B, and return to Tab A, your stack is restored.



Synthesis: "Flutter MapChat" Flow



1. Permissions

User taps "Nearby." App checks and requests `Permission.location`, showing a rationale first.



2. Routing

User taps a friend. App calls `context.go('/chat/alice_id')`. `go_router` builds the chat screen.



3. WebSockets

ChatScreen connects to `WebSocketManager` to send/receive messages in real-time.